Aula 06 - Aprenda Python - Definindo Funções

Rodrigo Attique

Desenvolvedor de Sistemas

Prof. Rodrigo Attique

Python é vida!

Funções

Funções

- Funções são blocos de código reutilizável, projetado para realizar uma tarefa específica, permitindo organizar, modularizar e evitar repetições no desenvolvimento de software.
- Ela pode receber dados de entrada (parâmetros), processá-los e retornar um resultado.
- Funções facilitam a manutenção e a legibilidade, dividindo sistemas complexos em partes menores.

Definindo funções

Podemos criar uma função que escreve a série de Fibonacci até um limite arbitrário:

```
>>> def fib(n):    # escreve série de Fibonacci menor que n
...     """Imprime uma série de Fibonacci menor que n."""
...     a, b = 0, 1
...     while a < n:
...         print(a, end=' ')
...         a, b = b, a+b
...     print()
...
>>> # Agora chamamos a função que acabamos de definir:
>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597
```

Definindo funções

A palavra reservada def inicia a definição de uma função. Ela deve ser seguida do nome da função e da lista de parâmetros formais entre parênteses. As instruções que formam o corpo da função começam na linha seguinte e devem ser indentados.

Palavra chave que define a função

Nome da Função

Função sem parametros

```
def nome_funcao():
    # alguma coisa
```

Escopo da função

Delimitador de Escopo

Palavra chave que define a função

Nome da Função

Função com 2 parâmetros

```
def soma(a, b):
    return a + b
```

Escopo da função que retorna soma dos parâmetros

Delimitador de Escopo

Chamada de Função

Uma definição de função associa o nome da função com o objeto função na tabela de símbolos atual. O interpretador reconhece o objeto apontado pelo nome como uma função definida pelo usuário. Outros nomes também podem apontar para o mesmo objeto função e também pode ser usados pra acessar a função:

```
>>> fib
<function fib at 10042ed0>
>>> f = fib
>>> f(100)
0 1 1 2 3 5 8 13 21 34 55 89
```

Funções com retorno

Conhecendo outras linguagens, pode-se questionar que fib não é uma função, mas um procedimento, pois ela não devolve um valor. Na verdade, mesmo funções que não usam a instrução return devolvem um valor, ainda que pouco interessante. Esse valor é chamado None (é um nome embutido). O interpretador interativo evita escrever None quando ele é o único resultado de uma expressão. Mas se quiser vê-lo pode usar a função print():

```
>>> fib(0)
>>> print(fib(0))
None
```

É fácil escrever uma função que retorna uma lista de números da série de Fibonacci, ao invés de exibi-los:

```
>>> def fib2(n):  # retorna série de  Fibonacci até n
...     """Retorna uma lista contendo a série de Fibonacci até n."""
...     resultado = []
...     a, b = 0, 1
...     while a < n:
...         resultado.append(a)    # veja abaixo
...         a, b = b, a+b
...     return resultado
...
>>> f100 = fib2(100)    # chama-o
>>> f100                # escreve o resultado
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Este exemplo demonstra novos recursos de Python

- A instrução return finaliza a execução e retorna um valor da função. return sem qualquer expressão como argumento retorna None.
- Atingir o final da função também retorna None.

Este exemplo demonstra novos recursos de Python

- A instrução **result.append(a)** chama um método do objeto lista **result**. **Um método é uma função que 'pertence' a um objeto**, e é chamada **obj.nomemetodo**, onde obj é um objeto qualquer (pode ser uma expressão), e nomemetodo é o nome de um método que foi definido pelo tipo do objeto.
- Tipos diferentes definem métodos diferentes.

Este exemplo demonstra novos recursos de Python

- Métodos de diferentes tipos podem ter o mesmo nome sem ambiguidade.
- (É possível definir seus próprios tipos de objetos e métodos, utilizando classes, veja em Classes) O método **append()**, mostrado no exemplo é definido para objetos do tipo lista; adiciona um novo elemento ao final da lista.
- Neste exemplo, ele equivale a **result = result + [a]**, só que mais eficiente.

Argumentos de Função

É possível definir funções com um número variável de argumentos. Existem três formas, que podem ser combinadas.

Argumentos com valor padrão

A mais útil das três é especificar um valor padrão para um ou mais argumentos.

Isso cria uma função que pode ser invocada com menos argumentos do que os que foram definidos.

Por exemplo:

Argumentos com valor padrão

```
def pergunta_ok(mensagem, tentativas=4, lembrete='Por favor, tente novamente!'):
    while True:
        resposta = input(mensagem)
        if resposta in {'s', 'sim', 'é'}:
            return True
        if resposta in {'n', 'não', 'nah'}:
            return False
        tentativas = tentativas - 1
        if tentativas < 0:
            raise ValueError('resposta inválida de usuário')
        print(lembrete)
```

Essa função pode ser chamada de várias formas

- fornecendo apenas o argumento obrigatório: ask_ok('Do you really want to quit?')
- fornecendo um dos argumentos opcionais: ask_ok('OK to overwrite the file?', 2)
- ou fornecendo todos os argumentos: ask_ok('OK to overwrite the file?', 2, 'Come on, only yes or no!')

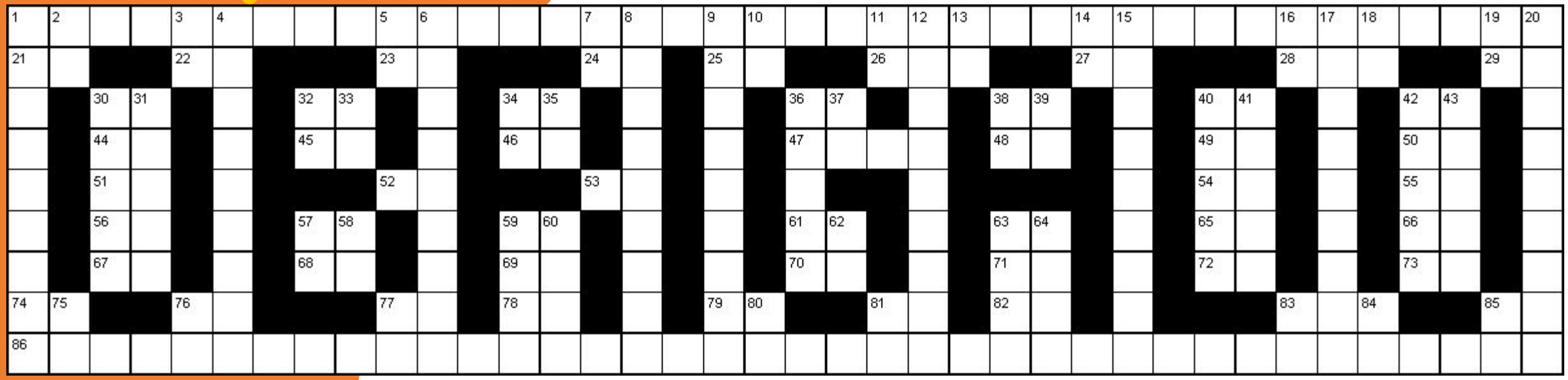
Pronto!

Ainda há muito o que aprender sobre Python, para nos aperfeiçoar é fundamental a prática de exercícios e situações que nos permitam evoluir.

Documentação de referência

https://docs.python.org/pt-br/3.14/tutorial/controlflow.html#defining-functions

Prof. Rodrigo Attique



Esta Foto de Autor Desconhecido está licenciado em CC BY-NC